

whispers)², after several rounds of communication, the original information may be quite distorted. Inspired by human social structures, we propose using structured communication to formulate the communication of agents. We establish a schema and format for each role and request that individuals provide the necessary outputs based on their specific role and context.

As shown in Figure 3, the Architect agent generates two outputs: the system interface design and a sequence flow diagram. These contain system module design and interaction sequences, which serve as important deliverables for Engineers. Unlike ChatDev (Zhao et al., 2023), agents in MetaGPT communicate through documents and diagrams (structured outputs) rather than dialogue. These documents contain all necessary information, preventing irrelevant or missing content.

Publish-Subscribe Mechanism Sharing information is critical in collaboration. For instance, Architects and Engineers often need to reference PRDs. However, communicating this information each time in a one-to-one manner, as indicated by previous work (Li et al., 2023; Zhao et al., 2023; Zhang et al., 2023a), can complicate the communication topology, resulting in inefficiencies.

To address this challenge, a viable approach is to store information in a global *message pool*. As shown in Figure 2 (left), we introduce a shared message pool that allows all agents to exchange messages directly. These agents not only *publish* their structured messages in the pool but also access messages from other entities transparently. Any agent can directly retrieve required information from the shared pool, eliminating the need to inquire about other agents and await their responses. This enhances communication efficiency.

Sharing all information with every agent can lead to information overload. During task execution, an agent typically prefers to receive only task-related information and avoid distractions through irrelevant details. Effective management and dissemination of this information play a crucial role. We offer a simple and effective solution-*subscription mechanism* (in Figure 2 (left)). Instead of relying on dialogue, agents utilize role-specific interests to extract relevant information. They can select information to follow based on their role profiles. In practical implementations, an agent activates its action only after receiving all its prerequisite dependencies. As illustrated in Figure 3, the Architect mainly focuses on PRDs provided by the Product Manager, while documents from roles such as the QA Engineer might be of lesser concern.

3.3 ITERATIVE PROGRAMMING WITH EXECUTABLE FEEDBACK

In daily programming tasks, the processes of debugging and optimization play important roles. However, existing methods often lack a self-correction mechanism, which leads to unsuccessful code generation. Previous work introduced non-executable code review and self-reflection (Zhao et al., 2023; Yao et al., 2022; Shinn et al., 2023; Dong et al., 2023). However, they still face challenges in ensuring code executability and runtime correctness.

Our first MetaGPT implementations overlooked certain errors during the review process, due to LLM hallucinations (Manakul et al., 2023). To overcome this, after initial code generation, we introduce an executable feedback mechanism to improve the code iteratively. More specifically, as shown in Figure 2, the Engineer is asked to write code based on the original product requirements and design.

This enables the Engineer to continuously improve code using its own historical execution and debugging memory. To obtain additional information, the Engineer writes and executes the corresponding unit test cases, and subsequently receives the test results. If satisfactory, additional development tasks are initiated. Otherwise the Engineer debugs the code before resuming programming. This iterative testing process continues until the test is passed or a maximum of 3 retries is reached.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETTING

Datasets We use two public benchmarks, HumanEval (Chen et al., 2021a) and MBPP (Austin et al., 2021), and a self-generated, more challenging software development benchmark named Soft-

²https://en.wikipedia.org/wiki/Chinese_whispers

wareDev: (1) HumanEval includes 164 handwritten programming tasks. These tasks encompass function specifications, descriptions, reference codes, and tests. (2) MBPP consists of 427 Python tasks. These tasks cover core concepts and standard library features and include descriptions, reference codes, and automated tests. (3) Our SoftwareDev dataset is a collection of 70 representative examples of software development tasks, each with its own task prompt (see Table 8). These tasks have diverse scopes (See Figure 5), such as mini-games, image processing algorithms, data visualization. They offer a robust testbed for authentic development tasks. Contrary to previous datasets (Chen et al., 2021a; Austin et al., 2021), SoftwareDev focuses on the engineering aspects. In the comparisons, we randomly select seven representative tasks for evaluation.

Evaluation Metrics For HumanEval and MBPP, we follow the unbiased version of Pass@ k as presented by (Chen et al., 2021a; Dong et al., 2023), to evaluate the functional accuracy of the top- k

generated codes: $\text{Pass@}k = \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$.

For SoftwareDev, we prioritize practical use and evaluate performance through human evaluations (A, E) or statistical analysis (B, C, D): **(A)** Executability: this metric rates code from 1 (failure/non-functional) to 4 (flawless). ‘1’ is for non-functional, ‘2’ for runnable but imperfect, ‘3’ for nearly perfect, and ‘4’ for flawless code. **(B)** Cost: the cost evaluations here include the (1) running time, (2) token usage, and (3) expenses. **(C)** Code Statistics: this includes (1) code files, (2) lines of code per file, and (3) total code lines. **(D)** Productivity: basically, it is defined as the number of token usage divided by the number of lines of code, which refers to the consumption of tokens per code line. **(E)** Human Revision Cost: refers to times of manual code corrections, which tackle problems like package import errors, incorrect class names, or incomplete reference paths. Typically, each correction involves up to 3 lines of code.

Baselines We compare our method with recent domain-specific LLMs in the code generation field, including AlphaCode (Li et al., 2022), Incoder (Fried et al., 2022), CodeGeeX (Zheng et al., 2023), CodeGen (Nijkamp et al., 2023), CodeX (Chen et al., 2021a), and CodeT (Chen et al., 2022) and general domain LLMs such as PaLM (Chowdhery et al., 2022), and GPT-4 (OpenAI, 2023). Several results of baselines (such as Incoder, CodeGeeX) are provided by Dong et al. (2023). In HumanEval and MBPP, we slightly modified the prompts to align with response format requirements. These modifications aim to address format-specific issues (i.e., Python problems). With the SoftwareDev benchmark, we provide a comprehensive comparison between MetaGPT, AutoGPT (Torantulino et al., 2023), LangChain (Chase, 2022) with Python Read-Eval-Print Loop (REPL) tool³, AgentVerse (Chen et al., 2023), and ChatDev (Qian et al., 2023).

4.2 MAIN RESULT

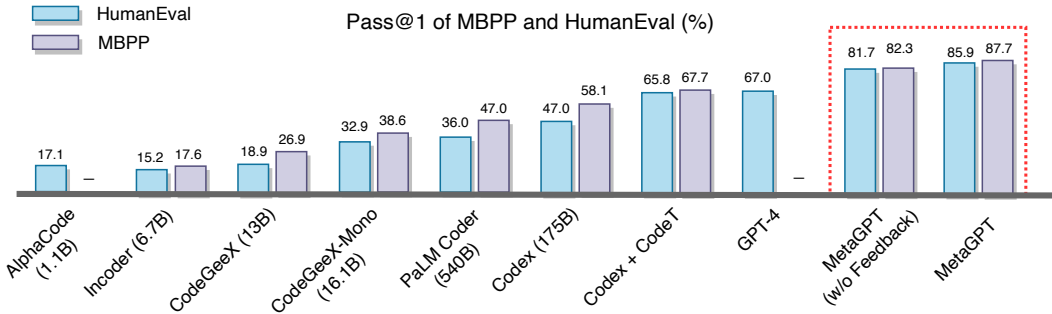


Figure 4: Pass rates on the MBPP and HumanEval with a single attempt.

Performance Figure 4 demonstrates that MetaGPT outperforms all preceding approaches in both HumanEval and MBPP benchmarks. When MetaGPT collaborates with GPT-4, it significantly improves the Pass@ k in the HumanEval benchmark compared to GPT-4. It achieves 85.9% and 87.7%

³https://en.wikipedia.org/wiki/Read-eval-print_loop

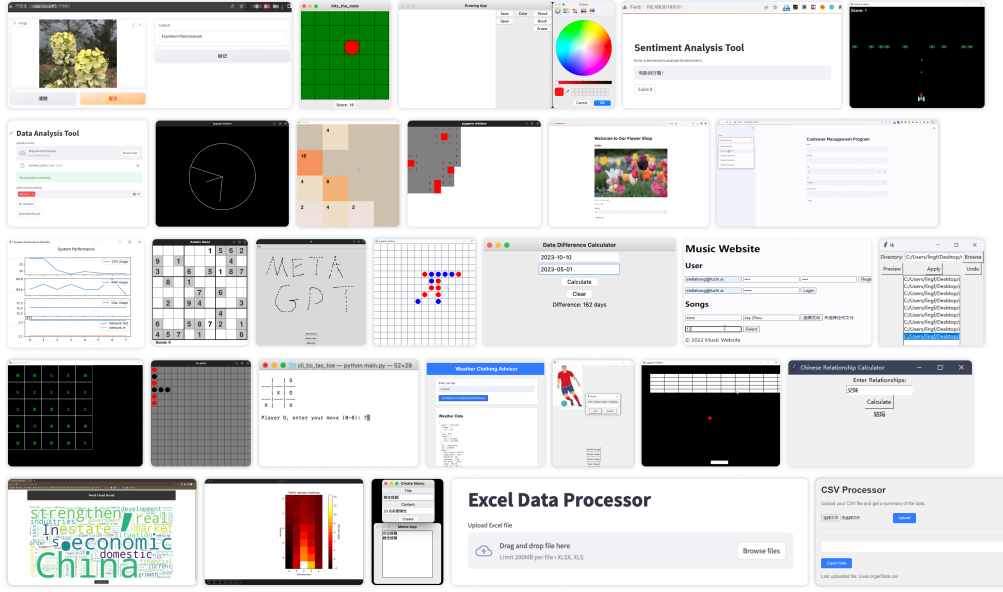


Figure 5: Demo softwares developed by MetaGPT.

in these two public benchmarks. Moreover, as shown in Table 1, MetaGPT outperforms ChatDev on the challenging SoftwareDev dataset in nearly all metrics. For example, considering the executability, MetaGPT achieves a score of 3.75, which is very close to 4 (flawless). Besides, it takes less time (503 seconds), clearly less than ChatDev. Considering the code statistic and the cost of human revision, it also significantly outperforms ChatDev. Although MetaGPT requires more tokens (24,613 or 31,255 compared to 19,292), it needs only 126.5/124.3 tokens to generate one line of code. In contrast, ChatDev uses 248.9 tokens. These results highlight the benefits of SOPs in collaborations between multiple agents. Additionally, we demonstrate the autonomous software generation capabilities of MetaGPT through visualization samples (Figure 5). For additional experiments and analysis, please refer to Appendix C.

Table 1: The statistical analysis on SoftwareDev.

Statistical Index	ChatDev	MetaGPT w/o Feedback	MetaGPT
(A) Executability	2.25	3.67	3.75
(B) Cost#1: Running Times (s)	762	503	541
(B) Cost#2: Token Usage	19,292	24,613	31,255
(C) Code Statistic#1: Code Files	1.9	4.6	5.1
(C) Code Statistic#2: Lines of Code per File	40.8	42.3	49.3
(C) Code Statistic#3: Total Code Lines	77.5	194.6	251.4
(D) Productivity	248.9	126.5	124.3
(E) Human Revision Cost	2.5	2.25	0.83

4.3 CAPABILITIES ANALYSIS

Compared to open-source baseline methods such as AutoGPT and autonomous agents such as AgentVerse and ChatDev, MetaGPT offers functions for software engineering tasks. As presented in Table 2, our framework encompasses a wide range of abilities to handle complex and specialized development tasks efficiently. Incorporating SOPs (e.g., role-play expertise, structured communication, streamlined workflow) can significantly improve code generation. Other baseline methods

Table 2: **Comparison of capabilities for MetaGPT and other approaches.** ‘✓’ indicates the presence of a specific feature in the corresponding framework, ‘✗’ its absence.

Framework Capability	AutoGPT	LangChain	AgentVerse	ChatDev	MetaGPT
PRD generation	✗	✗	✗	✗	✓
Tenical design generation	✗	✗	✗	✗	✓
API interface generation	✗	✗	✗	✗	✓
Code generation	✓	✓	✓	✓	✓
Precompilation execution	✗	✗	✗	✗	✓
Role-based task management	✗	✗	✗	✓	✓
Code review	✗	✗	✓	✓	✓

Table 3: **Ablation study on roles.** ‘#’ denotes ‘The number of’, ‘Product’ denotes ‘Product manager’, and ‘Project’ denotes ‘Project manager’. ‘✓’ indicates the addition of a specific role. ‘Revisions’ refers to ‘Human Revision Cost’.

Engineer	Product	Architect	Project	#Agents	#Lines	Expense	Revisions	Executability
✓	✗	✗	✗	1	83.0	\$ 0.915	10	1.0
✓	✓	✗	✗	2	112.0	\$ 1.059	6.5	2.0
✓	✓	✓	✗	3	143.0	\$ 1.204	4.0	2.5
✓	✓	✗	✓	3	205.0	\$ 1.251	3.5	2.0
✓	✓	✓	✓	4	191.0	\$ 1.385	2.5	4.0

can easily integrate SOP-like designs to improve their performance, similar to injecting chain-of-thought (Wei et al., 2022) in LLMs.

4.4 ABLATION STUDY

The Effectiveness of Roles To understand the impact of different roles on the final results, we perform two tasks that involve generating effective code and calculating average statistics. When we exclude certain roles, unworkable codes are generated. As indicated by Table 3, the addition of roles different from just the Engineer consistently improves both revisions and executability. While more roles slightly increase the expenses, the overall performance improves noticeably, demonstrating the effectiveness of the various roles.

The Effectiveness of Executable Feedback Mechanism As shown in Figure 4, adding executable feedback into MetaGPT leads to a significant improvement of 4.2% and 5.4% in Pass@1 on HumanEval and MBPP, respectively. Besides, Table 1 shows that the feedback mechanism improves feasibility (3.67 to 3.75) and reduces the cost of human revisions (2.25 to 0.83). These results illustrate how our designed feedback mechanism can produce higher-quality code. Additional quantitative results of MetaGPT and MetaGPT without executable feedback are shown in Table 4 and Table 9.

5 CONCLUSION

This work introduces MetaGPT, a novel meta-programming framework that leverages SOPs to enhance the problem-solving capabilities of multi-agent systems based on Large Language Models (LLMs). MetaGPT models a group of agents as a simulated software company, analogous to simulated towns (Park et al., 2023) and the Minecraft Sandbox in Voyager (Wang et al., 2023a). MetaGPT leverages role specialization, workflow management, and efficient sharing mechanisms such as message pools and subscriptions, rendering it a flexible and portable platform for autonomous agents and multi-agent frameworks. It uses an executable feedback mechanism to enhance code generation quality during runtime. In extensive experiments, MetaGPT achieves state-of-the-art performance on multiple benchmarks. The successful integration of human-like SOPs inspires future research on human-inspired techniques for artificial multi-agent systems. We also view our work as an early attempt to regulate LLM-based multi-agent frameworks. See also the **outlook** (Appendix A).

Acknowledgement

We thank Sarah Salhi, the Executive Secretary of KAUST AI Initiative, and Yuhui Wang, Postdoctoral Fellow at the KAUST AI Initiative, for helping to polish some of the text. We would like to express our gratitude to Wenyi Wang, a PhD student at the KAUST AI Initiative, for providing comprehensive feedback on the paper and for helping to draft the outlook (Appendix A) with Mingchen. We also thank Zongze Xu, the vice president of DeepWisdom, for providing illustrative materials for AgentStore.

Author Contributions

Sirui Hong conducted most of the experiments and designed the executable feedback module. She also led the initial version of the write-up, supported by Ceyao Zhang, and also by Jinlin Wang and Zili Wang. Mingchen Zhuge designed the self-improvement module, discussed additional experiments, and led the current write-up. Jiaqi Chen helped with the MBPP experiments, outlined the methods section, and contributed to the current write-up. Xiawu Zheng provided valuable guidance, reviewed and edited the paper. Yuheng Cheng contributed to the evaluation metric design and HumanEval experiments. Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Lingfeng Xiao helped with the MBPP experiments and comparisons to open-source baseline methods. Chenyu Ran created most of the illustrative figures. Chenglin Wu is the CEO of DeepWisdom, initiated MetaGPT, made the most significant code contributions to it, and advised this project. Jürgen Schmidhuber, Director of the AI Initiative at KAUST and Scientific Director of IDSIA, advised this project and helped with the write-up.

REFERENCES

- Elif Akata, Lion Schulz, Julian Coda-Forno, Seong Joon Oh, Matthias Bethge, and Eric Schulz. Playing repeated games with large language models. *arXiv preprint*, 2023.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.
- Anton Bakhtin, Noam Brown, Emily Dinan, Gabriele Farina, Colin Flaherty, Daniel Fried, Andrew Goff, Jonathan Gray, Hengyuan Hu, et al. Human-level play in the game of diplomacy by combining language models with strategic reasoning. *Science*, 2022.
- Robert Balzer. A 15 year perspective on automatic programming. *TSE*, 1985.
- R.M. Belbin. *Team Roles at Work*. Routledge, 2012. URL <https://books.google.co.uk/books?id=MHIQBAAQBAJ>.
- Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. *arXiv preprint*, 2023.
- Harrison Chase. LangChain. <https://github.com/hwchase17/langchain>, 2022.
- Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests, 2022.
- Jiaqi Chen, Yuxian Jiang, Jiachen Lu, and Li Zhang. S-agents: self-organizing agents in open-ended environment. *arXiv preprint*, 2024.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021a.